

INTERNSHIP PROJECT REPORT

**DESIGN AND VERIFICATION
OF 32-BIT ALU**

Submitted by
DHANANJAYA L
USN: 1JS24EC040
JSS ACADEMY OF TECHNICAL EDUCATION, BENGALURU

Internship Organization
Vaidsys Technologies

Internship Period
14th August 2026 – 13th September 2026

Academic Year 2026-27

DECLARATION

I, DHANANJAYA L (USN: 1JS24EC040), hereby declare that the internship project report entitled “Design and Verification of 32-bit ALU” is a record of the work carried out during the internship at Vaidsys Technologies from 14th August 2026 to 13th September 2026. The report has been prepared for academic submission to JSS Academy of Technical Education, Bengaluru.

The design results, simulation waveforms, RTL schematics, synthesis views, utilization information, timing summary and power analysis included in this report are based on the Vivado project developed and evaluated during the internship.

Place: Bengaluru

Date: 09/09/2026

Signature of Intern
DHANANJAYA L

ACKNOWLEDGEMENT

I express my sincere gratitude to Vaidsys Technologies for providing the opportunity to undertake this internship and work on the project “Design and Verification of 32-bit ALU.” The internship provided practical exposure to RTL design, Verilog HDL, testbench development, functional simulation, synthesis and FPGA implementation using AMD Vivado.

I also thank the faculty and management of JSS Academy of Technical Education, Bengaluru, for their guidance and academic support. I am grateful to everyone who contributed directly or indirectly to the successful completion of this project.

ABSTRACT

The Arithmetic Logic Unit (ALU) is one of the fundamental combinational blocks in a digital processor. It performs arithmetic, logical, shifting and comparison operations on binary operands. This internship project presents the design and verification of a 32-bit ALU using Verilog HDL and the AMD Vivado design environment.

The developed ALU accepts two 32-bit inputs, A and B, together with a 4-bit operation-select input. It supports addition, subtraction, bitwise AND, OR, XOR, NOT, logical left shift, logical right shift and comparison. A Verilog testbench applies different operand and select combinations so that each supported operation can be verified through behavioral simulation.

The design was elaborated, synthesized and implemented for the selected FPGA device in Vivado. The supplied results show an RTL structure containing arithmetic and logical blocks feeding a multiplexer, a successful multi-operation functional waveform, synthesis/implementation views, resource utilization, timing analysis and power estimation. The utilization report shows 315 Slice LUTs, 87 Slices, 315 LUTs used as logic and 100 bonded I/O resources. The timing summary explicitly states that no user-specified timing constraints are present; therefore, no verified maximum clock frequency is claimed. The power report has a low confidence level and is treated as an estimate rather than a measured power value.

The project demonstrates the complete basic RTL workflow from architecture definition and HDL coding through simulation, synthesis, implementation and report analysis.

Keywords

32-bit ALU, Verilog HDL, RTL Design, Functional Verification, Vivado, FPGA, Synthesis, Utilization, Timing Analysis, Power Analysis.

TABLE OF CONTENTS

Section	Title
1	Introduction
2	Problem Statement and Objectives
3	Background and ALU Architecture
4	Design Methodology
5	Functional Verification
6	RTL Synthesis and Implementation
7	Results and Analysis
8	Learning Outcomes
9	Limitations and Scope for Enhancement
10	Conclusion
11	References
Appendix A	Operation Summary

1. INTRODUCTION

1.1 Overview

An Arithmetic Logic Unit is a digital combinational circuit that performs arithmetic and logical transformations on binary data. In a processor datapath, the ALU is responsible for operations such as addition, subtraction, Boolean logic, shifts and comparisons. The width of the ALU determines the number of bits processed in one operation; therefore, a 32-bit ALU processes 32-bit operands and generates a 32-bit result.

The purpose of this project is to implement a compact 32-bit ALU in Verilog HDL and verify that the selected operation is correctly reflected at the output. The project also introduces the standard FPGA development flow: RTL entry, elaboration, behavioral simulation, synthesis, implementation and analysis of generated reports.

1.2 Need for the Project

Studying an ALU provides a practical bridge between digital logic theory and processor architecture. The project combines combinational logic design, HDL coding, verification and FPGA tool usage in a single design. It also demonstrates how a high-level RTL description is transformed by synthesis into FPGA logic resources.

1.3 Project Scope

- Design a 32-bit combinational ALU using Verilog HDL.
- Support arithmetic, Boolean, shift and comparison operations.
- Develop a testbench and verify the operations using behavioral simulation.
- Perform RTL elaboration and synthesis in Vivado.
- Inspect the synthesized/implemented design and FPGA device view.
- Study utilization, timing and power reports.
- Document the design flow, results, limitations and possible enhancements.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Processors require efficient arithmetic and logical operations with optimized use of hardware resources. The problem addressed in this internship project is to design a 32-bit ALU capable of performing a defined set of arithmetic and logic functions and to verify its behavior through simulation and FPGA design analysis.

2.2 Objectives

- Design a 32-bit ALU.
- Support arithmetic and logical operations.
- Perform functional simulation using a Verilog testbench.
- Generate and inspect RTL and synthesized schematics.
- Perform synthesis and implementation in Vivado.
- Analyze FPGA resource utilization.
- Study timing and power reports without overstating unconstrained or low-confidence results.
- Prepare technical documentation of the complete design and verification flow.

2.3 Functional Requirements

ALU_Sel	Operation	Function
---------	-----------	----------

0000	Addition	Result = A + B
0001	Subtraction	Result = A - B
0010	AND	Result = A & B
0011	OR	Result = A B
0100	XOR	Result = A ^ B
0101	NOT	Result = ~A
0110	Shift Left	Result = A << B[4:0]
0111	Shift Right	Result = A >> B[4:0]
1000	Comparison	Result = 1 when A < B, otherwise 0

3. BACKGROUND AND ALU ARCHITECTURE

3.1 Input and Output Interface

Signal	Width	Direction	Description
A	32 bits	Input	First operand
B	32 bits	Input	Second operand / shift amount source
ALU_Sel	4 bits	Input	Selects the required ALU operation
Result	32 bits	Output	Result of the selected operation

3.2 Architectural Principle

At the RTL level, the required arithmetic and logic functions are described behaviorally. The operation-select input controls which expression drives the 32-bit Result output. Conceptually, the design consists of parallel function blocks whose outputs are selected by a multiplexer.

The elaborated schematic supplied from Vivado confirms this structure: addition, subtraction, AND, OR, XOR, inversion, left shift, right shift and comparison logic feed a final RTL multiplexer controlled by ALU_Sel.

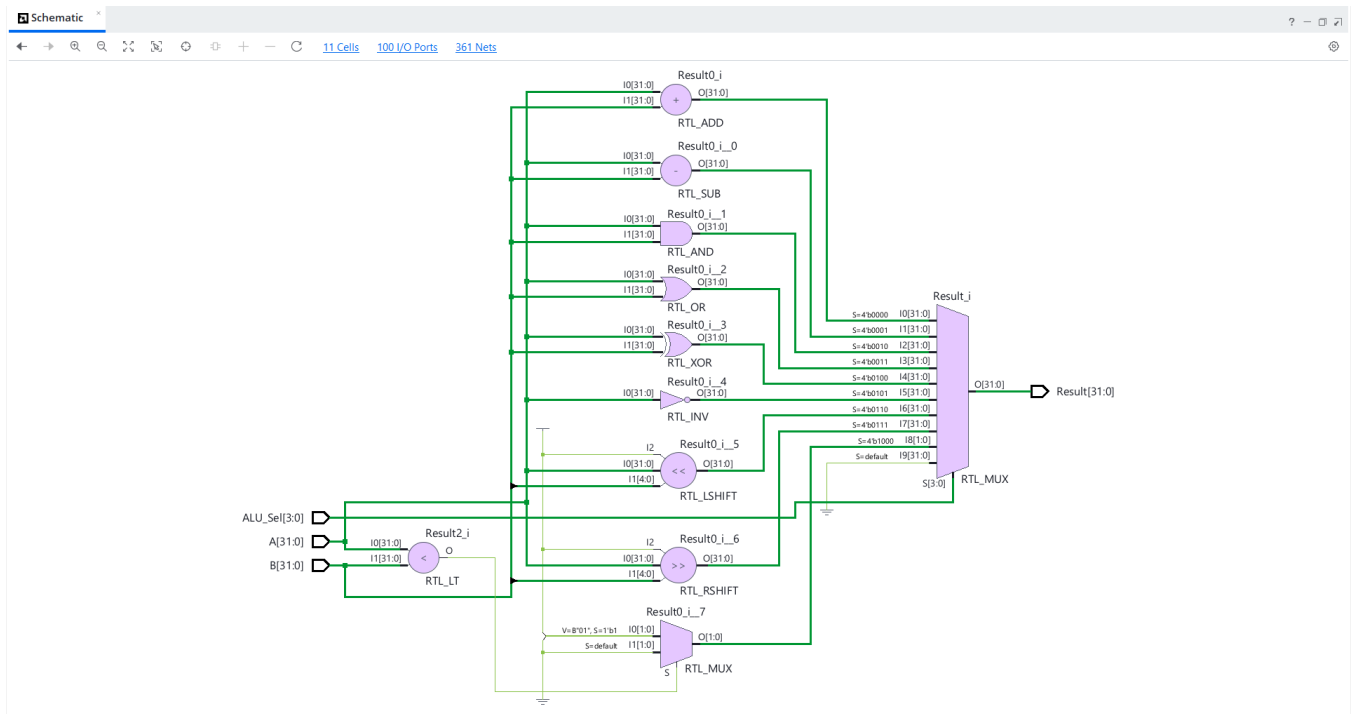


Figure 3.1: Detailed RTL schematic of the 32-bit ALU

4. DESIGN METHODOLOGY

4.1 Development Tools

Tool / Technology	Purpose
Verilog HDL	RTL description of the ALU and testbench development
AMD Vivado 2026.1	Project creation, simulation, RTL analysis, synthesis, implementation and reports
Vivado Simulator	Behavioral functional verification and waveform inspection
ModelSim / GTKWave	Listed as alternative simulation/waveform tools in the project scope
Synopsys Design Compiler	Optional synthesis tool listed in the project scope; the supplied results are from Vivado

4.2 Design Flow

1. Study the required ALU operations and define the input/output interface.
2. Create the Verilog design module for the 32-bit ALU.
3. Implement operation selection using combinational behavioral logic.
4. Create a Verilog testbench with different values of A, B and ALU_Sel.
5. Run behavioral simulation and inspect the Result waveform.
6. Open the elaborated design and inspect the RTL schematic.
7. Run synthesis and inspect the synthesized design.
8. Run implementation for the selected FPGA device.
9. Generate utilization, timing, power and DRC reports.
10. Document observations and identify limitations or enhancement opportunities.

4.3 Verification Strategy

The testbench changes ALU_Sel across the supported operation codes and applies representative operand values. The output is then inspected in the waveform window. This approach verifies that the combinational selection logic produces the expected result for each operation.

5. FUNCTIONAL VERIFICATION

5.1 Behavioral Simulation

Behavioral simulation was performed before relying on synthesis or implementation results. The supplied 90 ns waveform exercises ALU_Sel values from 0 through 8 and demonstrates the output response for the supported operations.

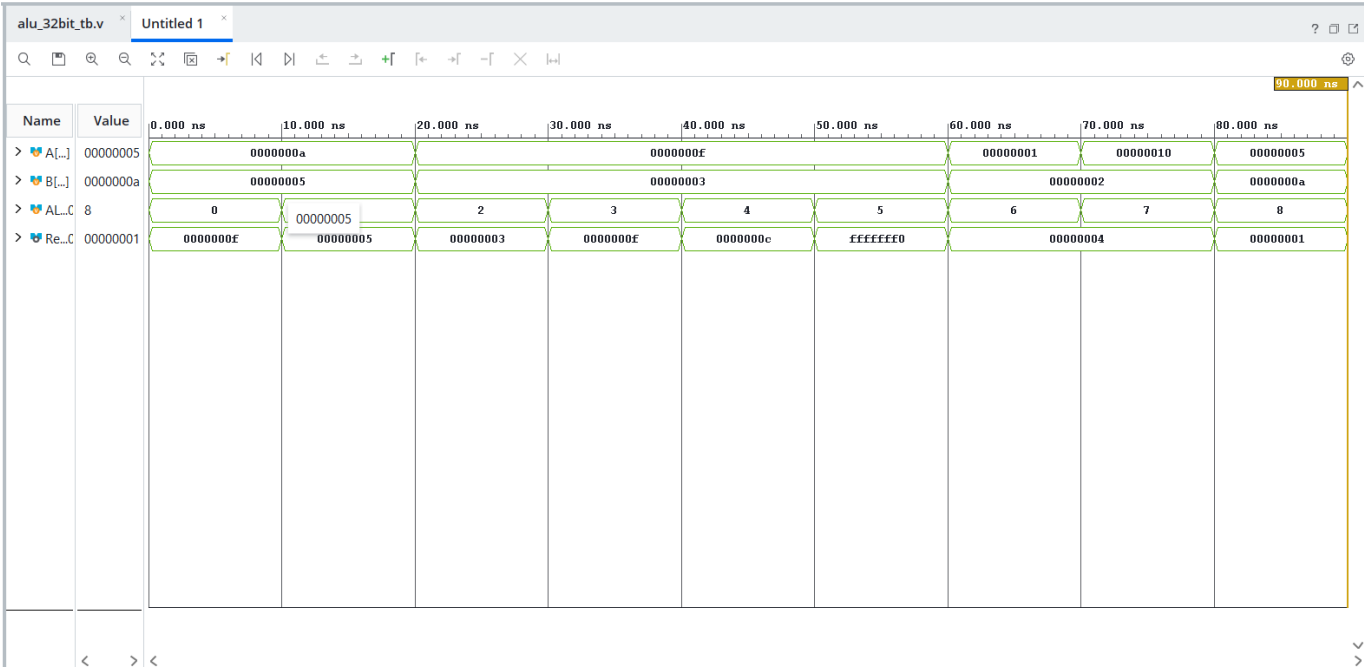


Figure 5.1: Behavioral simulation waveform covering ALU operations 0 through 8

5.2 Waveform Interpretation

Time Region	ALU_Sel	Observed Example	Interpretation
0-10 ns	0	A=0x0A, B=0x05, Result=0x0F	Addition: 10 + 5 = 15
10-20 ns	1	A=0x0A, B=0x05, Result=0x05	Subtraction: 10 - 5 = 5
20-30 ns	2	A=0x0F, B=0x03, Result=0x03	AND
30-40 ns	3	A=0x0F, B=0x03, Result=0x0F	OR
40-50 ns	4	A=0x0F, B=0x03, Result=0x0C	XOR
50-60 ns	5	A=0x0F, Result=0xFFFFFFFF0	Bitwise NOT of A
60-70 ns	6	A=0x01, B=0x02, Result=0x04	Left shift by 2
70-80 ns	7	A=0x10, B=0x02, Result=0x04	Right shift by 2
80-90 ns	8	A=0x05, B=0x0A,	Comparison: 5 < 10 is true

		Result=0x01	
--	--	-------------	--

The waveform therefore provides direct functional evidence for all nine specified operations. The values shown above are read from the supplied Vivado waveform and are consistent with the operation mapping represented in the RTL schematic.

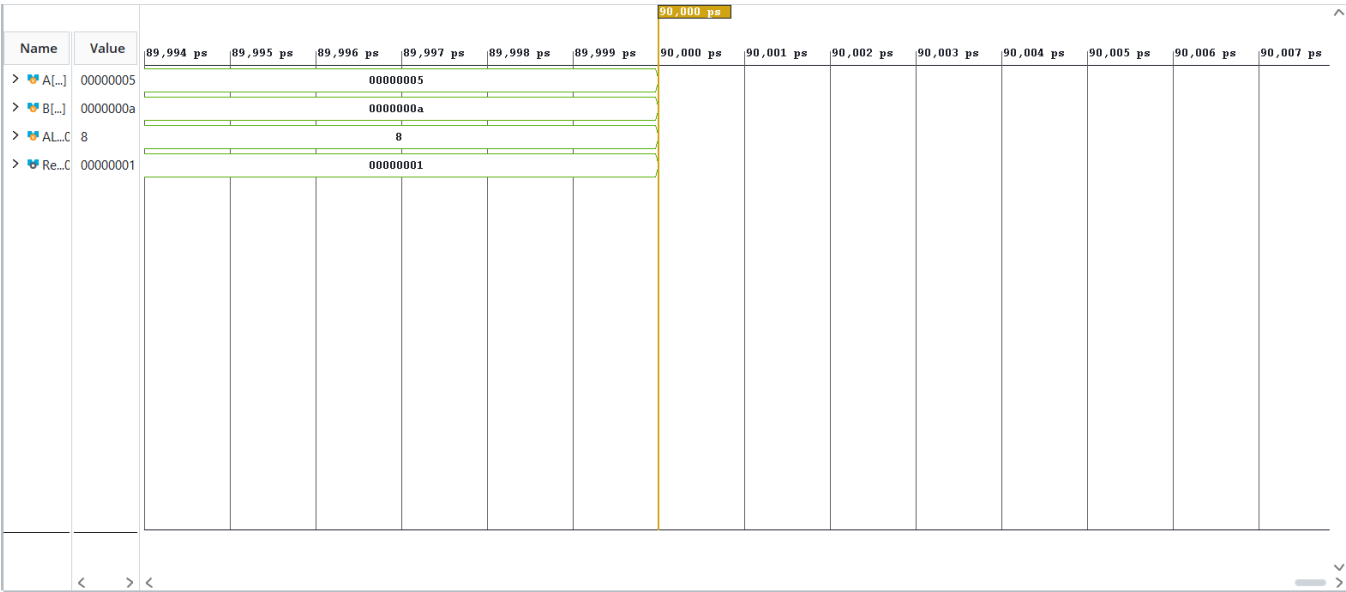


Figure 5.2: Focused comparison test showing A=5, B=10 and Result=1

6. RTL SYNTHESIS AND IMPLEMENTATION

6.1 RTL Elaboration

RTL elaboration converts the Verilog description into a structural representation while preserving the intended register-transfer behavior. The supplied elaborated design reports 11 cells, 100 I/O ports and 361 nets. The schematic clearly identifies the arithmetic, logical, shift and comparison blocks and the final multiplexer.

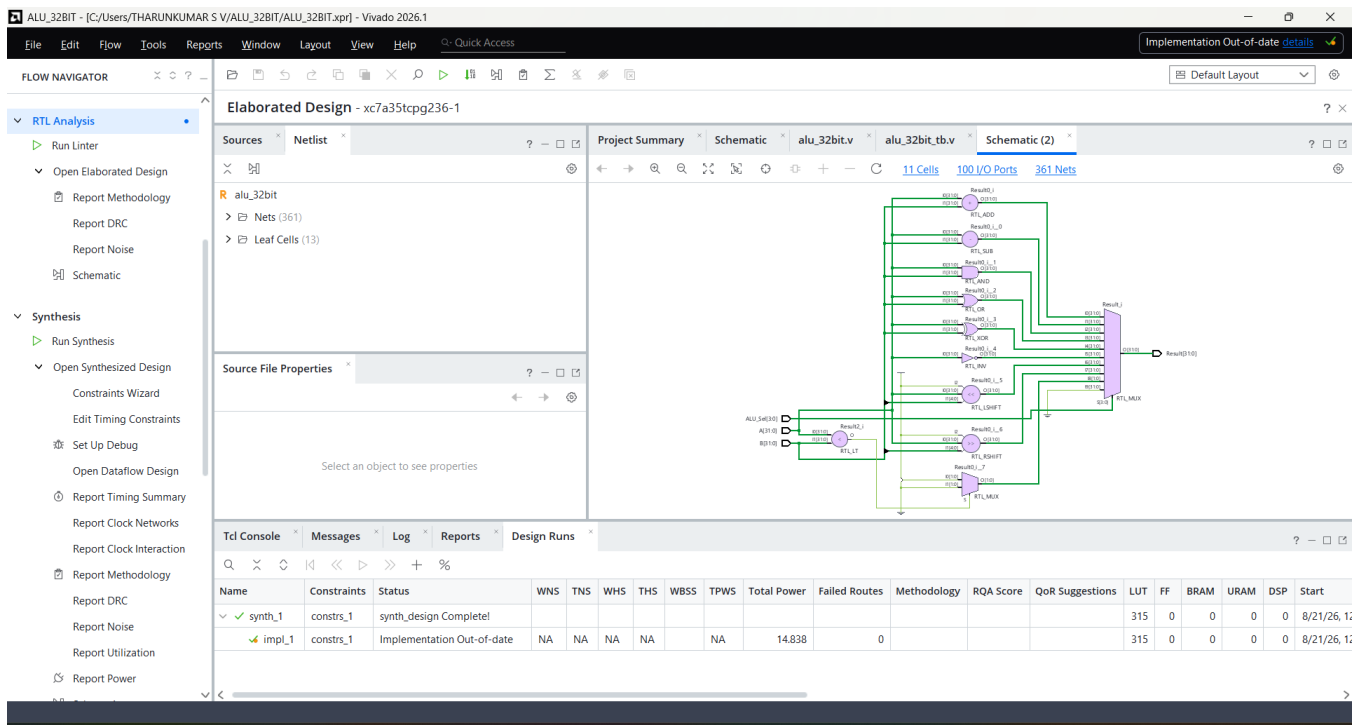


Figure 6.1: Vivado elaborated design showing ALU function blocks and output multiplexer

6.2 Synthesis

Synthesis maps the RTL description to FPGA-oriented logic primitives. The supplied synthesized/implemented schematic is substantially more detailed than the RTL schematic because high-level operators are decomposed into lower-level logic and routing structures.

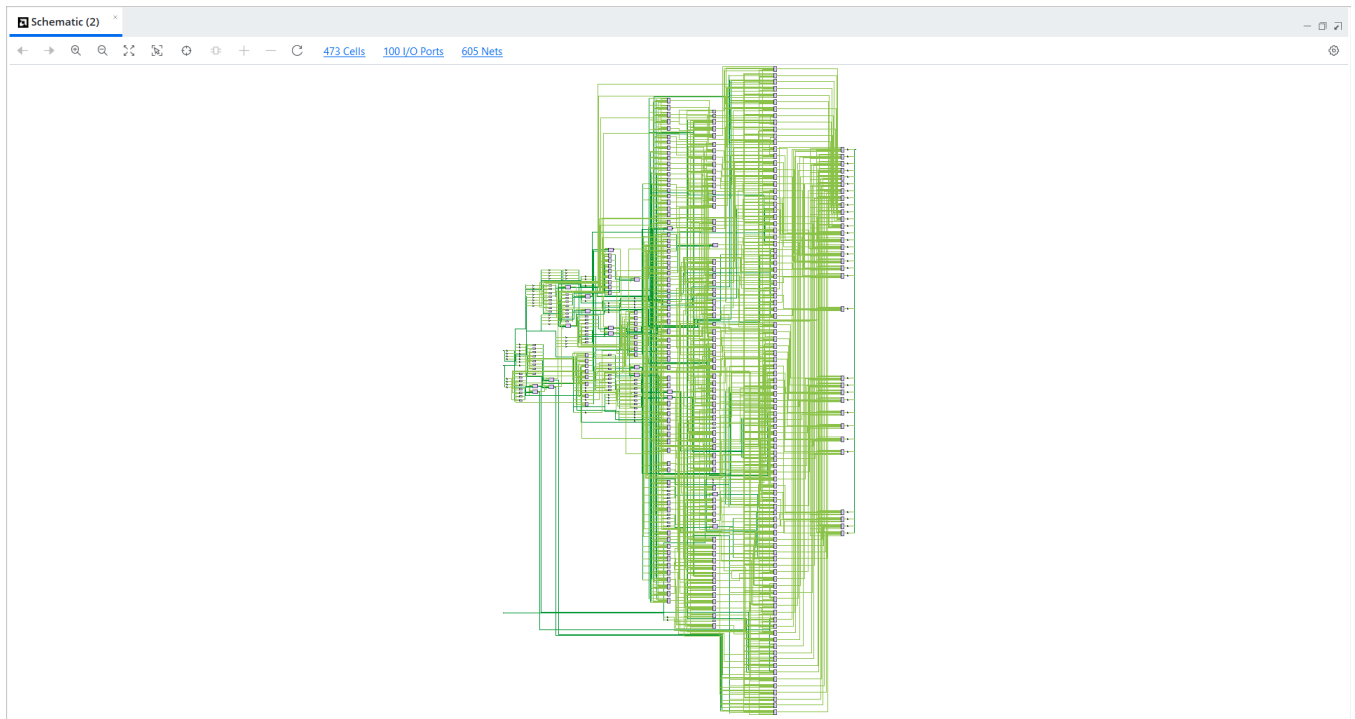


Figure 6.2: Synthesized/implemented logic schematic

6.3 FPGA Implementation

Implementation places and routes the synthesized logic on the selected FPGA device. The supplied device view shows the physical FPGA fabric and the region in which logic associated with the ALU design is placed.

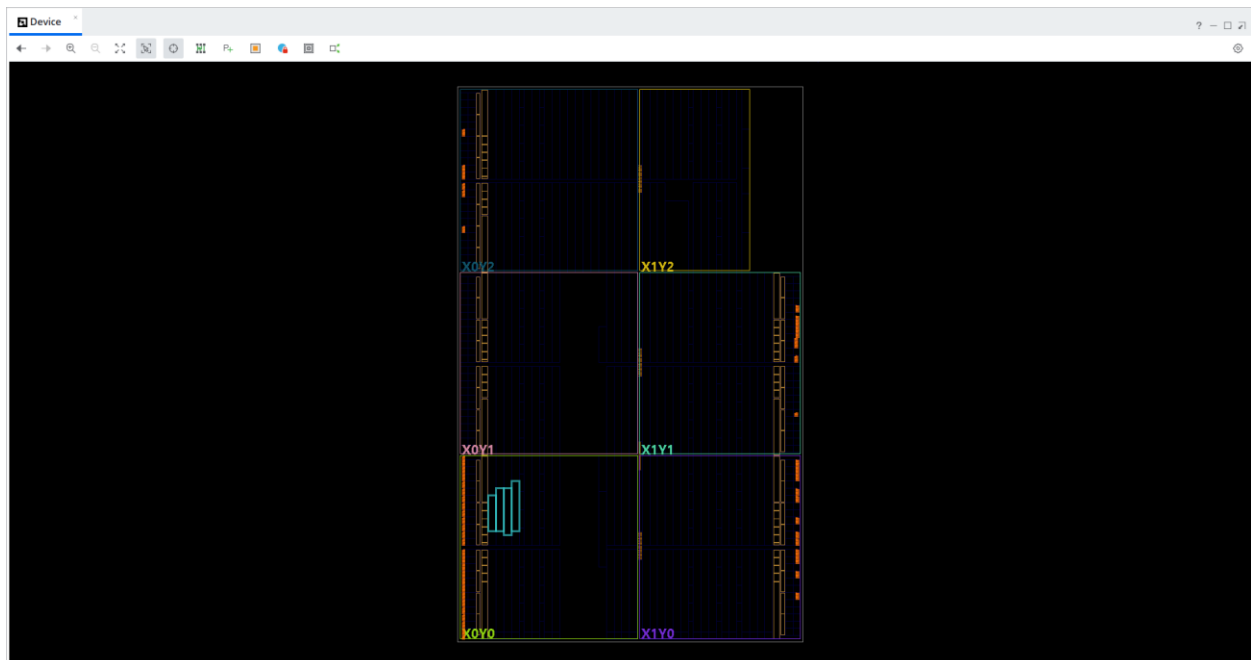


Figure 6.3: FPGA device view after implementation

7. RESULTS AND ANALYSIS

7.1 Resource Utilization

The implementation utilization report provides the clearest area/resource result available from the supplied project evidence.

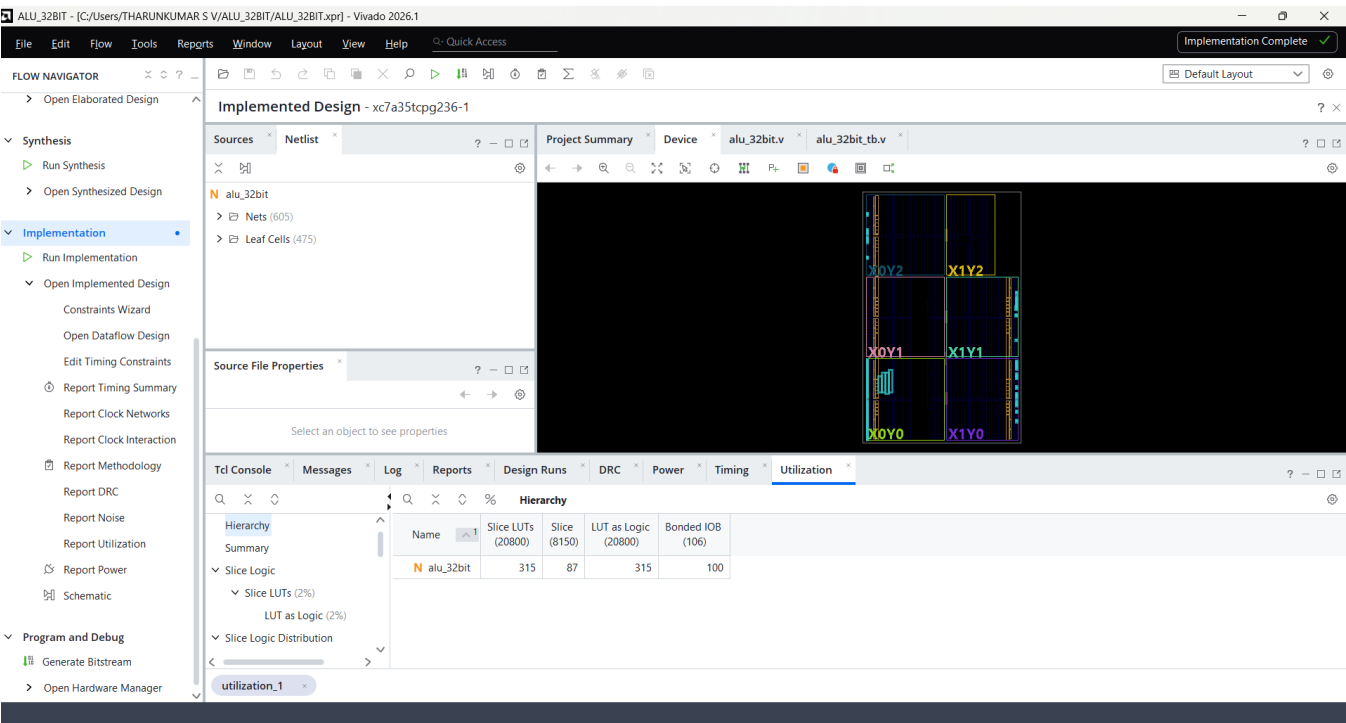


Figure 7.1: Vivado resource utilization report

Resource	Used	Available	Approx. Utilization
Slice LUTs	315	20,800	1.51%
Slices	87	8,150	1.07%
LUT as Logic	315	20,800	1.51%
Bonded IOB	100	106	94.34%

The logic utilization is low relative to the available LUT and Slice capacity, indicating that the combinational ALU occupies a small fraction of the selected FPGA logic fabric. The bonded I/O usage is high because the design exposes two 32-bit inputs, one 4-bit select input and one 32-bit output, giving 100 logical external I/O signals.

7.2 Timing Analysis

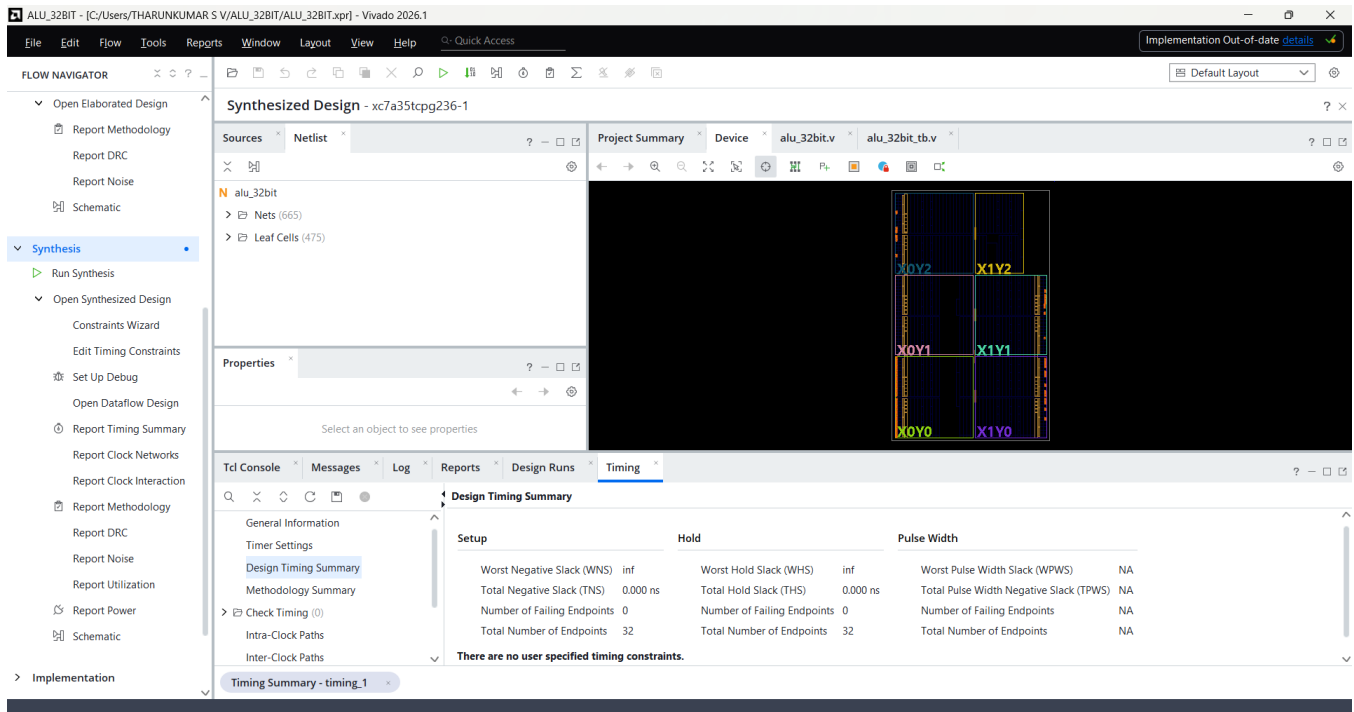


Figure 7.2: Vivado design timing summary

The timing summary reports zero failing endpoints, but it also explicitly states: “There are no user specified timing constraints.” The displayed WNS and WHS values are therefore not sufficient to claim that a specific clock period or maximum operating frequency has been achieved. This is expected for the current purely combinational ALU project unless meaningful input/output or clock-related timing constraints are defined.

For a future pipelined or registered ALU, a clock constraint and appropriate input/output delays should be added before making frequency or timing-closure claims.

7.3 Power Analysis

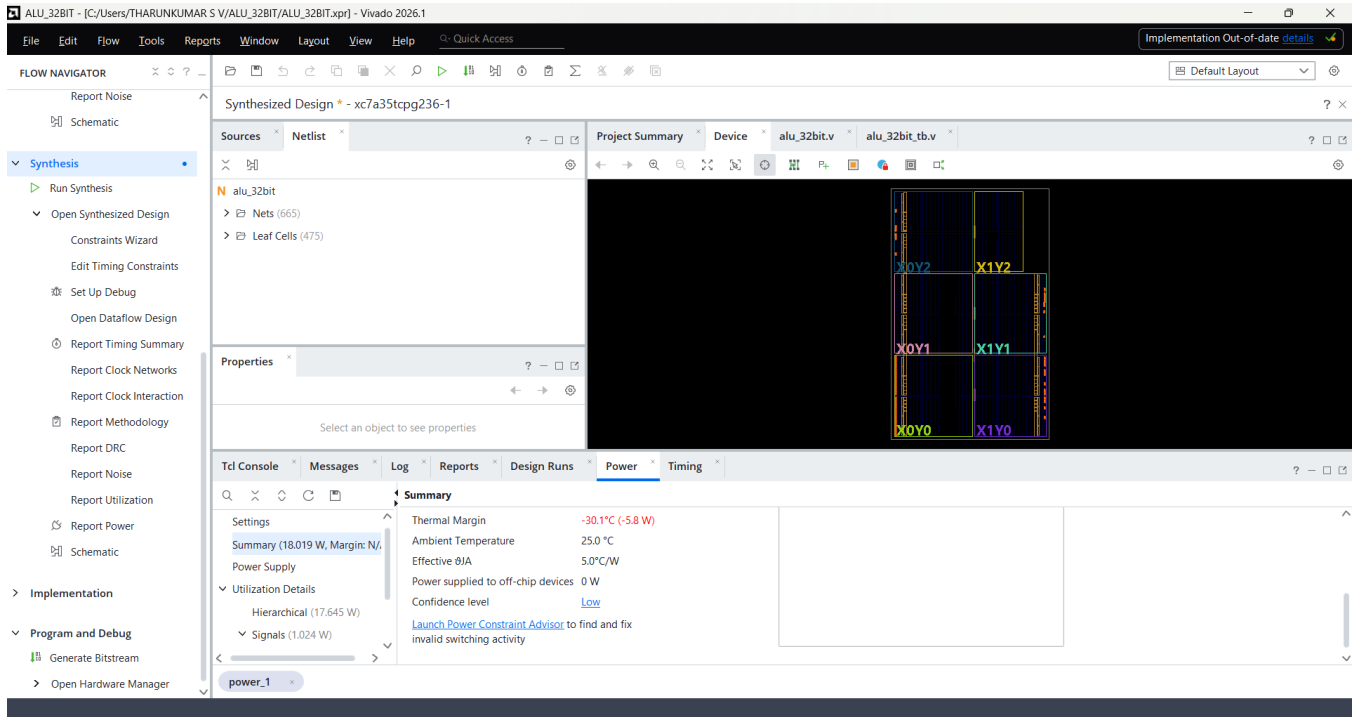


Figure 7.3: Vivado power analysis summary

The supplied power summary displays an estimated total power of approximately 18.019 W and a low confidence level. It also indicates invalid or incomplete switching activity information. Consequently, this value must be treated as a tool estimate under incomplete activity assumptions, not as measured hardware power consumption.

A more reliable power analysis would require realistic switching activity, a finalized implementation, correct environmental/device settings and, ideally, comparison with measurements from physical hardware.

7.4 Design Rule Check

A final DRC run performed after correcting the project constraints returned “No Violations Found.” This indicates that the selected DRC rule set did not detect remaining design-rule violations in that run. Earlier warnings related to unspecified I/O standards and configuration voltage were addressed during the project workflow before the final DRC check.

7.5 Overall Result

Project Requirement	Status	Evidence
32-bit ALU design	Completed	RTL schematic and Verilog project
Addition / subtraction	Verified	Behavioral waveform
AND / OR / XOR / NOT	Verified	Behavioral waveform
Shift left / right	Verified	Behavioral waveform
Comparison	Verified	Behavioral waveform
Functional simulation	Completed	90 ns Vivado waveform
RTL synthesis	Completed	Synthesized design / schematic

FPGA implementation	Completed	Implemented device view
Area/resource analysis	Completed	Utilization report
Timing analysis	Generated with limitation	No user-specified timing constraints
Power analysis	Generated with limitation	Low-confidence estimate
DRC	Completed	Final run: No Violations Found

8. LEARNING OUTCOMES

The internship project provided practical experience across the basic digital hardware development lifecycle. The major learning outcomes are:

- Understanding the role and architecture of a processor ALU.
- Writing synthesizable combinational RTL in Verilog HDL.
- Using a 4-bit control input to select multiple 32-bit operations.
- Developing a testbench and interpreting hexadecimal waveform results.
- Using Vivado RTL Analysis to inspect the elaborated design.
- Running synthesis and understanding the transformation from RTL operators to FPGA logic.
- Running implementation and examining the FPGA device view.
- Reading resource utilization reports and relating I/O count to the module interface.
- Recognizing the importance of timing constraints before making performance claims.
- Recognizing the limitations of low-confidence power estimates.
- Using DRC results to identify and correct FPGA constraint issues.
- Preparing technical documentation from simulation and implementation evidence.

9. LIMITATIONS AND SCOPE FOR ENHANCEMENT

9.1 Current Limitations

- The ALU is combinational and does not contain pipeline registers.
- The supplied timing analysis has no user-specified timing constraints, so maximum frequency is not established.
- The supplied power estimate has low confidence because realistic switching activity is not fully specified.
- The current report documents simulation and FPGA implementation views; physical board-level validation is not shown in the supplied evidence.

9.2 Future Enhancements

- Introduce pipelining to improve achievable throughput and enable meaningful clock-based timing analysis.
- Add status flags such as Zero, Carry, Borrow, Overflow and Negative.
- Extend the ALU with multiplication, division or arithmetic shifts if required by the target processor.
- Develop a self-checking or assertion-based testbench for automatic pass/fail verification.
- Use constrained-random test vectors to improve functional coverage.
- Provide realistic switching activity for improved power estimation.
- Implement and validate the ALU on an FPGA development board.
- Explore low-power techniques and compare area, delay and power across alternative architectures.

10. CONCLUSION

The internship project “Design and Verification of 32-bit ALU” successfully demonstrates the design of a 32-bit combinational Arithmetic Logic Unit in Verilog HDL and its verification using Vivado. The ALU supports nine required operations: addition, subtraction, AND, OR, XOR, NOT, left shift, right shift and comparison.

The behavioral waveform verifies the expected functionality across ALU_Sel values 0 through 8. RTL elaboration confirms the intended architecture of parallel operation blocks followed by output selection. Synthesis and implementation demonstrate that the RTL can be mapped to the selected FPGA device. The utilization report shows modest logic usage, while the timing and power reports highlight important engineering lessons: timing performance should not be claimed without appropriate constraints, and low-confidence power estimates should not be treated as physical measurements.

Overall, the project achieved the stated learning objectives in RTL design, testbench development, functional verification, synthesis flow, FPGA implementation and report interpretation, and it provides a suitable foundation for future pipelined, low-power and hardware-validated ALU designs.

11. REFERENCES

- [1] AMD Vivado Design Suite documentation and in-tool reports used during the project.
- [2] Verilog HDL concepts used for combinational RTL design and testbench development.
- [3] Project specification: “Design and Verification of 32-bit ALU,” supplied for the internship work.

APPENDIX A - OPERATION SUMMARY

Select	Operation	Representative Test	Expected Result
0 / 0000	ADD	10 + 5	15 (0x0000000F)
1 / 0001	SUB	10 - 5	5 (0x00000005)
2 / 0010	AND	0x0F & 0x03	0x00000003
3 / 0011	OR	0x0F 0x03	0x0000000F
4 / 0100	XOR	0x0F ^ 0x03	0x0000000C
5 / 0101	NOT	~0x0000000F	0xFFFFFFFF0
6 / 0110	Shift Left	1 << 2	4
7 / 0111	Shift Right	16 >> 2	4
8 / 1000	Comparison	5 < 10	1

Appendix Note

The operation mapping and representative values above are based on the supplied RTL schematic and behavioral simulation waveform. No source-code listing is reproduced because the complete Verilog source text was not supplied as report content in this conversation.